Integrated Systems Engineering Framework for Chaos-Aware Forest Cover Type Prediction

Nicolás Martínez Pineda (20241020098)
Anderson Danilo Martínez Bonilla (20241020107)
Gabriel Esteban Gutiérrez Calderón (20221020003)
Jean Paul Contreras Talero (20242020131)
Systems analysis and design
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

***Abstract*—This paper integrates the results of a multi-stage course project on forest cover type prediction using the Roosevelt National Forest dataset. Building on four analytical and architectural workshops and a previous chaos-aware architecture paper, we present a unified systems engineering framework that goes from problem characterization and sensitivity analysis to production-grade deployment and simulation. The proposed pipeline combines domain-driven feature engineering, ensemble learning, uncertainty quantification, and a hybrid machine-learning / cellular automata simulation.**

The architecture explicitly addresses ecological threshold sensitivity, soil-type sparsity, data drift, and operational robustness. Each workshop contributed a different viewpoint: the first characterized the problem and identified chaotic behaviours in elevation bands; the second proposed a seven-layer conceptual architecture; the third refined this design into a four-layer cloud-native implementation with explicit governance mechanisms; and the fourth implemented a simulation framework that couples a LightGBM classifier with a cellular automata model.

Experimental results show that the ensemble model can reach accuracies around 95% on the Kaggle benchmark while maintaining chaos-aware monitoring near critical elevation bands. The cellular automata simulation reproduces plausible successional patterns and highlights the importance of uncertainty control in transition zones. The overall framework illustrates how systems engineering principles—modularity, loose coupling, high cohesion, and fault-tolerance—can be combined with ecological knowledge to build decision-support tools for sustainable forest management.

***Index Terms*—Kaggle, Forest Cover Type, Systems Engineering, Chaos, Uncertainty Quantification, Simulation, Cellular Automata.**

I. Introduction

Forest ecosystems are complex adaptive systems where vegetation patterns emerge from the interaction of topography, climate, disturbance history, and soil properties. Accurately predicting forest cover type is essential for land-use planning, biodiversity conservation, fire management, and climate adaptation. However, the underlying processes are often nonlinear and exhibit chaotic behaviour: small variations in elevation or slope orientation can trigger abrupt shifts in species composition, especially near ecological transition zones.

The Kaggle *Forest Cover Type Prediction* competition provides a well-structured benchmark for this problem. The Roosevelt National Forest dataset contains 15 120 observations described by 56 cartographic variables, including ten continuous topographic features, four wilderness-area indicators, and forty soil-type categories. The goal is to classify each $30 \times 30\,\mathrm{m}$ cell into one of seven forest cover types. From a purely machine-learning perspective, this is a multi-class supervised classification problem. From a systems engineering perspective, it is also an opportunity to analyse the interactions among environmental drivers, technical design choices, and operational constraints.

In practical scenarios, a forest cover prediction system must operate under uncertainty, limited data, and changing environmental conditions. Models trained on historical data are often deployed in conditions that differ from the original distribution, and their predictions can be particularly fragile around ecological thresholds. Traditional competition-oriented pipelines tend to ignore these issues; they focus on maximizing accuracy without explicitly addressing robustness, drift detection, or interpretability. The course project that motivates this paper was designed to move beyond that mindset.

Throughout the semester, the project evolved through four workshops and a previous catch-up paper. The first workshop focused on systems analysis, identifying the main elements of the problem, their relationships, and regions of high sensitivity and potential chaos. The second workshop translated this understanding into a seven-layer architecture that covers ingestion, validation, feature engineering, modelling, uncertainty quantification, monitoring, and deployment. The third workshop consolidated this design into a four-layer production architecture aligned with quality and governance standards. Finally, the fourth workshop implemented a simulation framework combining a LightGBM classifier with a cellular automata model to explore forest succession dynamics under controlled perturbations.

The main goal of this paper is to present a single, coherent description of the resulting system. Specifically, we aim

to: (i) summarize the problem and dataset from a systems-engineering point of view, (ii) describe the integrated architecture that emerged from the workshops, (iii) discuss the performance, robustness, and chaos-aware properties of the solution, and (iv) highlight lessons learned that are relevant for both engineering practice and future research.

The rest of the paper is organized as follows. Section II describes the dataset, tools, and methods used in the four workshops and in the implementation of the final system. Section III presents the experimental results and discusses the behaviour of the model and the simulation framework. Section IV summarizes the conclusions and outlines directions for future work.

II. Methods and Materials

A. Dataset and Problem Definition

The Roosevelt National Forest dataset contains 15 120 samples, each describing a 30×30 m cell with 56 features. Ten variables are continuous topographic attributes: elevation, aspect, slope, three hillshade indices (at different times of day), horizontal distance to hydrology, vertical distance to hydrology, horizontal distance to roadways, and horizontal distance to fire ignition points. Four variables are one-hot encoded wilderness areas. The remaining forty variables are one-hot encoded soil-type indicators. The target variable is a categorical label representing one of seven forest cover types: Spruce/Fir, Lodgepole Pine, Ponderosa Pine, Cottonwood/Willow, Aspen, Douglas-fir, and Krummholz.

Elevations in the dataset range approximately from 1 859 to 3 858 m. During the first workshop, the team inspected the joint distribution of elevation and cover type to identify characteristic zones. Three broad climatic regions were recognized: foothill (approximately below 2 400 m), montane–subalpine (2 400–3 200 m), and high subalpine–alpine (above 3 200 m). Within these regions, narrower elevation bands were observed where cover-type frequencies change abruptly. These bands were characterized as *ecological thresholds* and were later treated as "chaos zones" because small changes in elevation around them can lead to large changes in cover type.

The soil-type variables introduce a different kind of complexity. With forty one-hot encoded categories and a limited number of samples, the representation is highly sparse. Many soil types appear in only a few samples, which increases dimensionality without necessarily adding robust predictive signal. This sparsity motivates both regularization in the models and aggregation strategies at the feature-engineering stage.

Formally, the problem can be described as learning a function $f : \mathbb{R}^{56} \to \{1, \ldots, 7\}$ that maps the feature vector of each cell to its cover type. Beyond accuracy, the system must provide calibrated probabilities, explicit uncertainty measures, and operational mechanisms to detect drift and handle failures.

B. Tools and Implementation Environment

The solution was implemented in Python using a set of widely adopted libraries. Data exploration and preprocessing were carried out with `pandas` and `NumPy`. The main ensemble models—Random Forest and XGBoost—were implemented using `scikit-learn` and `xgboost`, while LightGBM was integrated through its dedicated Python interface. Hyperparameter tuning used randomized search and Bayesian optimization techniques. Cross-validation was stratified by cover type to maintain class proportions in all folds.

The preprocessing pipeline was built with the `ColumnTransformer` and `Pipeline` abstractions from `scikit-learn`, which allow clean separation of numerical and categorical transformations. The final pipeline was exported as a reusable artifact and served by an inference component designed under a microservices approach. Although the implementation ran in a laboratory environment, the architecture was designed to be deployable in container-based infrastructure such as Docker and Kubernetes.

The cellular automata simulation used a custom implementation in Python, representing the forest as a two-dimensional grid. Each cell stored its elevation, aspect, and current cover type. Local rules were evaluated at each time step to compute candidate transitions, which were then modulated by the LightGBM model acting as an environmental regulator.

C. Systems Analysis and Sensitivity (Workshop 1)

The first workshop focused on understanding the problem as a system of interacting components rather than as a purely statistical task. The analysis identified three conceptual layers: input, processing, and output. The input layer is formed by the 56 features grouped into numerical drivers, wilderness zone indicators, and soil-type categories. The processing layer corresponds to the machine-learning algorithms and the additional mechanisms used to handle uncertainty and robustness. The output layer represents predicted cover types and derived products such as maps or risk indicators.

Key relationships were identified between elevation, slope, aspect, and cover type. Elevation acts as a master driver structuring temperature and precipitation regimes. Aspect modifies solar exposure, so north-facing slopes tend to be cooler and moister than south-facing slopes. Slope influences soil depth, erosion, and water retention. Distance to hydrology and fire points embeds the influence of water availability and disturbance history. These relationships were initially analysed through correlation matrices and conditional distributions, revealing non-monotonic patterns and interactions.

Complexity analysis highlighted potential single points of failure and regions of high sensitivity. Soil-type sparsity was recognized as a risk for overfitting. The geographic coverage of the dataset (essentially one forest system) introduced a risk of poor generalization to other regions. The absence of temporal variation limited the ability to model climate trends or succession explicitly. Most importantly, the elevation thresholds acted as regions where small changes in input features could produce large changes in output. This behaviour is analogous to the sensitive dependence on initial conditions studied in chaos theory and motivated the introduction of chaos-aware mechanisms in the architecture.

D. Seven-Layer Architecture Design (Workshop 2)

The second workshop translated the previous analysis into a seven-layer conceptual architecture. Each layer encapsulates a clearly defined set of responsibilities and communicates with adjacent layers through explicit data contracts.

The first layer, *Data Ingestion and Collection*, integrates raw cartographic data from official sources into a unified dataset. It ensures consistent coordinate systems, handles basic format conversions, and records metadata such as data source and acquisition date. The second layer, *Data Validation and Quality Assurance*, performs range checks, detects missing values, and identifies multivariate outliers. Instead of silently discarding anomalous records, the system attaches quality flags that can be used later for filtering or for detailed analysis.

The third layer, *Feature Engineering*, implements domain-driven transformations. Examples include elevation-zone binning, encoding aspect with sine and cosine components, and aggregating soil types into broader categories associated with similar physical properties. Interaction features combine elevation with distance to hydrology or slope to capture known ecological effects. This layer also implements normalization and encoding strategies required by the models.

The fourth layer, *Model Training and Ensemble Integration*, trains several tree-based models with complementary characteristics. Random Forest provides robust baselines and interpretable feature importance. XGBoost captures complex nonlinear interactions through gradient boosting. LightGBM offers computational efficiency and sparse-feature handling. Models are trained with cross-validation, and their outputs are combined using a weighted voting strategy tuned on a validation set.

The fifth layer, *Prediction and Uncertainty Quantification*, generates class probabilities and decomposes uncertainty. Aleatoric uncertainty is estimated from the shape of the predicted probability distribution for each sample, while epistemic uncertainty is inferred from the disagreement among base models. Samples located in chaos zones receive an additional uncertainty amplification factor to reflect their intrinsic instability.

The sixth layer, *Monitoring and Drift Detection*, tracks distributional changes in input features and in predicted classes. Metrics such as the Population Stability Index and divergences between historical and current feature distributions are computed periodically. Significant deviations trigger alerts and can lead to automatic actions, such as blocking new data from entering the training process or forcing retraining of the models. This layer reduces the risk of silent degradation when the environment changes.

The seventh layer, *Deployment and Operations*, focuses on delivering the model as a reliable service. It defines interfaces for batch and real-time prediction, manages model and preprocessor versioning, and specifies backup strategies. Fault-tolerance mechanisms include health checks, circuit breakers, and graceful degradation: if the full ensemble is temporarily unavailable, a simpler but robust baseline model can respond while the system recovers.

E. Four-Layer Production Architecture (Workshop 3)

In the third workshop, the conceptual seven-layer pipeline was consolidated into a four-layer cloud-native architecture better suited to production constraints. The objective was to keep the essential functions while reducing complexity and ensuring coherence with quality and governance practices.

1) Data Gate: The *Data Gate* layer combines ingestion and validation in a single fault-isolated entry point. Incoming datasets are checked against a schema that specifies feature names, types, and valid ranges. The layer also verifies that categorical values correspond to known wilderness and soil codes. If the input violates the schema, the request is rejected and an alert is generated. For valid inputs, the Data Gate performs train–test splitting, ensuring that class proportions and elevation distributions are preserved. By enforcing these checks early, the Data Gate acts as a circuit breaker that prevents corrupt or shifted data from propagating downstream.

2) Preprocessing and Feature Engineering: The second layer implements the preprocessing and feature engineering pipeline. It is configured declaratively, using a column transformer that applies standardized transformations to groups of features. Numerical variables are scaled, aspect is transformed trigonometrically, and soil types are optionally aggregated into broader categories. Class imbalance is mitigated through techniques such as class-weight adjustment or synthetic oversampling, depending on experimental results. The entire pipeline is serialized and reused both at training time and during inference, guaranteeing that the models always see inputs with the same preprocessing logic.

3) Model Training, Evaluation, and Governance: The third layer is responsible for model training and evaluation, including explicit quality gates. Several candidate configurations of Random Forest, XGBoost, and LightGBM are explored using hyperparameter search. For each configuration, cross-validation metrics such as accuracy, macro F1-score, and calibration error are computed. A model is only considered a deployment candidate if it outperforms a reference baseline and satisfies predefined thresholds. The governance component records metadata about each experiment: training data version, hyperparameters, performance metrics, and author. This information supports traceability and reproducibility.

4) Inference Core with Uncertainty and Chaos-Aware Logic: The fourth layer hosts the inference core. It exposes an API that receives feature vectors (or small batches) and returns predicted cover types together with class probabilities and uncertainty indicators. The ensemble logic combines outputs from the three base models; if one model fails or is temporarily unavailable, the system degrades gracefully by using the remaining models. The core also implements chaos-aware logic: samples located in elevation bands previously identified as thresholds automatically receive higher uncertainty scores, and the system can mark them as "needs human review" according to a configurable policy.

F. Simulation Framework (Workshop 4)

The fourth workshop implemented a hybrid simulation framework with two complementary scenarios. The goal was not only to test the trained models, but also to explore dynamic forest behaviour and to validate the consistency between the data-driven model and a simple process model.

1) Scenario 1: Data-Driven ML Simulation: The first scenario focuses on the data-driven pipeline itself. The full preprocessing and ensemble inference chain is applied to a held-out test set and, optionally, to synthetic grids that interpolate elevation and aspect values within the observed ranges. Performance metrics include overall accuracy, macro-averaged F1-score, per-class precision and recall, and calibration curves. Stratified analyses are performed by elevation band and wilderness area to identify regions where the model behaves differently.

This scenario also investigates the relationship between chaos zones and uncertainty. By grouping samples according to their distance to critical elevation thresholds, we can compare entropy and model disagreement across groups. The expectation is that uncertainty will increase near thresholds, validating the chaos-aware components of the architecture.

2) Scenario 2: Event-Based Cellular Automata Simulation: The second scenario uses a cellular automata model to simulate forest succession on a 100×100 grid. Each cell has a fixed elevation and aspect (sampled from realistic distributions) and a dynamic cover type. The automaton evolves in discrete time steps according to local transition rules inspired by ecological processes:

- Neighbourhood influence: cells tend to adopt the most frequent cover type in their Moore neighbourhood, reflecting seed dispersal and local competition.
- Disturbance events: with a small probability, a disturbance (such as fire or logging) resets the cover type of a cell to a pioneer state, more typical of early successional stages.
- Environmental suitability: transitions that are inconsistent with the underlying elevation and aspect are penalized.

The trained LightGBM model acts as an environmental regulator. For each proposed transition, the automaton consults the model to obtain the most plausible cover type given the cell's features. If the proposed transition is compatible with the model's prediction, it is accepted with high probability. If it conflicts strongly with the model's distribution, the transition can be rejected or modified towards the model's most probable class. Inside chaos zones, the transition acceptance is more flexible to reflect the higher instability of those areas. In this way, the simulation explores emergent spatial patterns while remaining constrained by data-driven knowledge.

III. Results and Discussion

A. Predictive Performance and Feature Relevance

Across experiments, the ensemble composed of Random Forest, XGBoost, and LightGBM consistently achieved accuracies around 95% on the Kaggle-style test splits. Macro-averaged F1-scores were slightly lower but still high, reflecting the impact of minority classes such as Cottonwood/Willow. Class-wise analysis showed that dominant classes like Lodgepole Pine and Spruce/Fir are easier to predict, while classes associated with narrow environmental niches exhibit higher error rates.

Feature-importance analyses derived from the tree-based models confirmed the central role of elevation as a master driver. Distance to hydrology, aspect, and certain soil groups also appeared among the most influential features. This is consistent with ecological expectations: elevation structures climate, water availability and topography shape moisture regimes, and soil type influences rooting conditions and nutrient availability. The fact that data-driven importance agrees with domain knowledge increases trust in the model.

B. Uncertainty and Chaos Awareness

Uncertainty analysis revealed systematic patterns. In homogeneous regions far from identified thresholds, entropy values were low and model disagreement was rare. In contrast, samples located within roughly $\pm 50\,\mathrm{m}$ of critical elevation bands exhibited higher entropy and more frequent disagreement among base models. This behaviour was observed consistently across cross-validation folds and test subsets.

The chaos-aware logic implemented in the inference core uses this behaviour to provide more informative outputs. For example, end users can define an uncertainty threshold to decide when predictions are automatically accepted and when they require review by experts. In areas with high intrinsic instability, the system tends to produce more "review" flags, which is desirable from a risk-management perspective. Rather than hiding uncertainty, the system exposes it in a structured way.

C. Robustness, Drift, and Governance

The transition from a conceptual seven-layer design to the four-layer production architecture significantly improved robustness. The Data Gate reduced the probability that corrupted or shifted inputs would contaminate the training process. In controlled experiments where some features were artificially perturbed, the Data Gate correctly flagged anomalous distributions and prevented those datasets from being treated as normal training data.

The presence of an explicit quality gate in the training layer also reduced the risk of performance regressions. New models were not deployed simply because they represented newer experiments; they had to demonstrate superior performance with respect to defined metrics. This approach is aligned with quality standards such as ISO 9000 and Capability Maturity Model Integration (CMMI), where process discipline and traceability are central.

Monitoring components, even in a prototype form, showed how drift could be detected. For example, when the elevation distribution of incoming samples was shifted towards higher values, the Population Stability Index increased beyond the usual range, triggering an alert. This signal would inform operators that the model is being used in conditions that

differ from those seen during training, and that retraining or adaptation strategies might be necessary.

D. Simulation Insights

The data-driven simulation in Scenario 1 confirmed that the pipeline is computationally efficient. Training times remained within practical limits for classroom and prototyping contexts, and inference latency for individual samples was compatible with real-time map generation. Repeated runs showed that performance metrics were stable across random seeds, suggesting that the ensemble is not excessively sensitive to specific splits.

The cellular automata simulation in Scenario 2 produced spatial patterns that are ecologically plausible when inspected visually. After several tens of generations, high-elevation regions tended to be dominated by Krummholz and Spruce/Fir, while mid-elevation zones contained mosaics of Lodgepole Pine, Aspen, and Douglas-fir. Low elevations were more frequently associated with Ponderosa Pine and mixed communities.

An important observation was the interaction between the automaton and the LightGBM regulator in chaos zones. In those bands, the automaton proposed more frequent changes, and the model sometimes overruled unrealistic transitions. As a result, the spatial patterns remained consistent with the empirical distributions observed in the dataset, but local fluctuations were more common. This behaviour is coherent with the idea that those zones are both dynamic and constrained by environmental conditions.

Quantitatively, agreement between the automaton's state and the model's predictions remained substantially above random-chance levels, even after many generations. At the same time, the number of corrections requested by the model was higher in chaos zones than in stable regions. This reinforces the role of the ML model as an environmentally informed regulator that helps keep the simulation within realistic boundaries.

E. Limitations and Lessons Learned

Despite its positive results, the framework has several limitations. First, the dataset covers only one forest system and one time period. Thus, generalization to other regions or to future conditions affected by climate change is not guaranteed. Second, the cellular automata rules are intentionally simple; they capture only a subset of the processes that drive real forest dynamics. Third, uncertainty quantification is limited by the use of ensemble disagreement; more formal Bayesian approaches could provide deeper insight into model confidence.

From a pedagogical point of view, the project demonstrated the value of linking systems engineering concepts with a concrete data-science task. The workshop structure forced the team to think in terms of elements, relationships, complexity, risk, and governance, rather than only in terms of accuracy and code. This perspective is essential when machine-learning models are used in socio-technical systems where errors and failures have real consequences.

IV. Conclusions

This paper presented an integrated systems engineering framework for forest cover type prediction, combining the outcomes of four workshops and a previous architecture paper into a single document. Starting from a systemic analysis of elements, relationships, sensitivity, and chaotic behaviour, we designed a multi-layer architecture that addresses data quality, feature engineering, ensemble modelling, uncertainty quantification, monitoring, and governance. The refined four-layer production architecture demonstrated how conceptual designs can be transformed into robust, cloud-native systems.

On the methodological side, the framework combines supervised learning with a cellular automata simulation, allowing both static prediction and dynamic exploration of successional trajectories. The results show that it is possible to achieve high predictive accuracy while still exposing uncertainty and respecting ecological complexity. Chaos-aware mechanisms around elevation thresholds provide more realistic expectations about model reliability and support safer decision-making.

Future work may extend this framework along several directions: incorporating temporal data to represent climate trends and succession more realistically; integrating additional environmental variables such as precipitation or temperature; experimenting with Bayesian or deep-learning models that offer alternative forms of uncertainty quantification; and validating the approach on other forest systems to study generalization. Another relevant extension is to link the model with economic or policy modules to evaluate the consequences of different management scenarios.

Overall, the project illustrates how systems engineering thinking can guide not only the implementation of a machine learning solution, but also its integration into a broader socio-technical context where interpretability, robustness, and governance are as important as raw accuracy.

Acknowledgment

The authors thank the course instructor and the Computer Engineering Program at Universidad Distrital Francisco José de Caldas for their guidance and support throughout the development of this project.

References

[1] Kaggle, "Forest Cover Type Prediction," accessed 2025. [Online]. Available: https://www.kaggle.com
[2] A. Saltelli *et al.*, *Global Sensitivity Analysis: The Primer*. Wiley, 2008.
[3] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. KDD*, 2016, pp. 785–794.
[5] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. NIPS*, 2017, pp. 3146–3154.